

Lecture 2



香港城市大學
City University of Hong Kong
Innovating into the Future

Basic of Probability Distributions

Topics in Generative AI

Dr. Jianyuan Guo

Fall 2025, Dept. CS, CityU

Overview



- What is a generative model
- Representing probability distributions
 - Curse of dimensionality
 - Graphical models (Bayesian networks)
 - Neural models

Acknowledgement

This presentation is based on materials from:

- [6.S978: Deep Generative Models] by [Kaiming He/MIT], [2024]
- [CS236 : Deep Generative Models] by [Stefano Ermon/Stanford], [2023]

The slides have been adapted and extended with additional content, examples, and illustrations for the purpose of this lecture.

Perception (Discriminative) vs. Generative models

discriminative

- “sample” $x \Rightarrow$ “label” y
- one desired output

generative

- “label” $y \Rightarrow$ “sample” x
- many possible outputs

discriminative

x



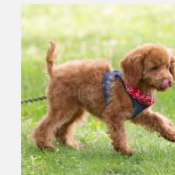
“dog”

y

generative

y

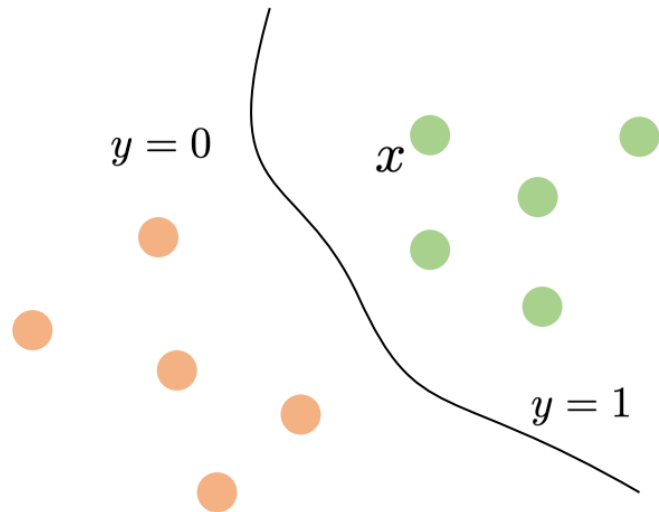
“dog”



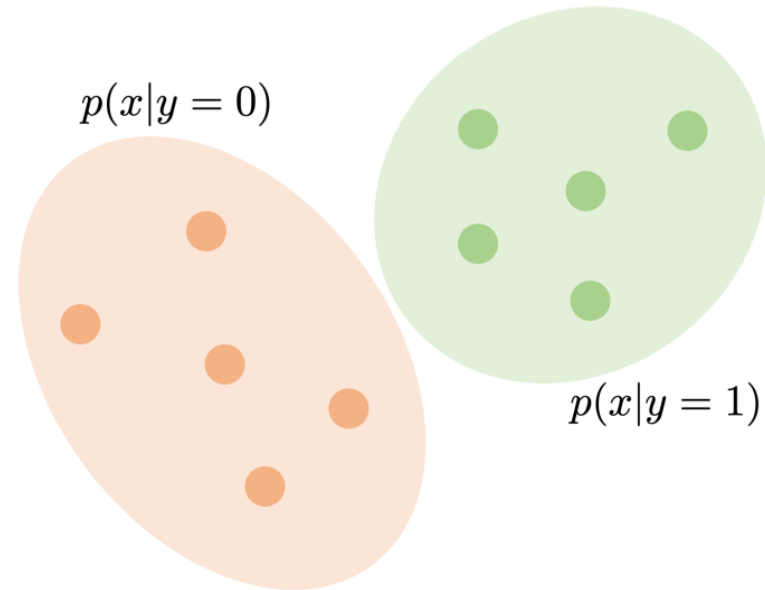
x

Discriminative vs. Generative models

discriminative $p(y|x)$



generative $p(x|y)$



What are Generative Models?

ChatGPT ▾

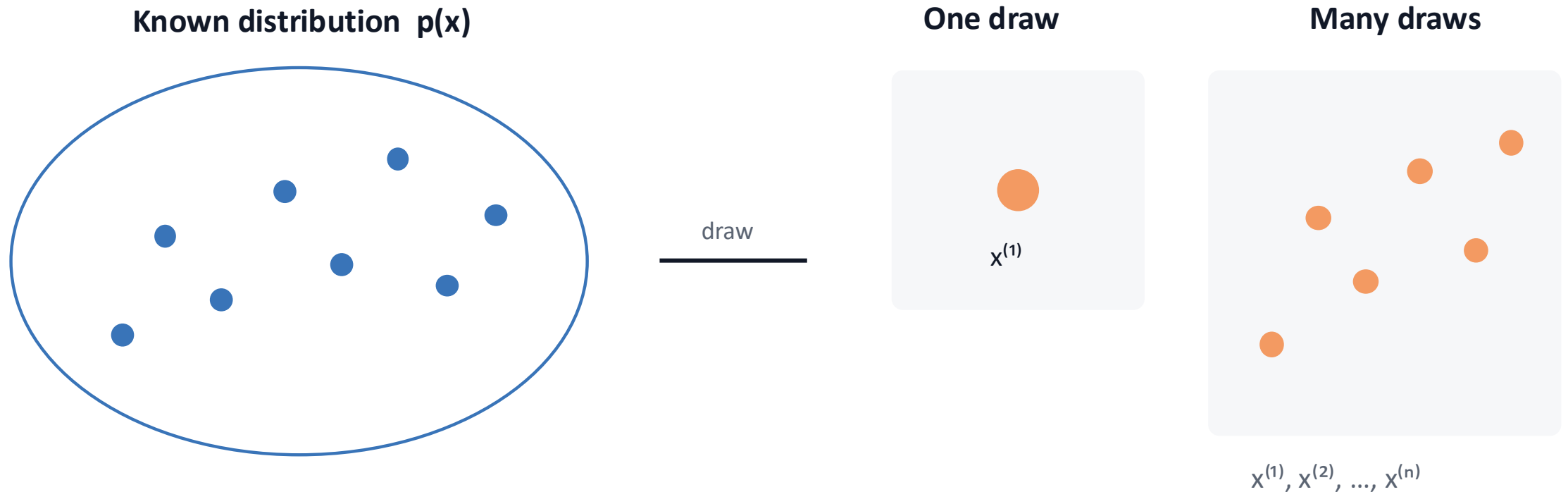
What are generative models?

Generative models are a class of machine learning models that **learn the underlying distribution of data** so they can generate new samples that resemble the training data.

In simple terms: instead of just learning to classify or predict (like *discriminative models*), generative models try to **model how the data was created**, allowing them to generate new, realistic-looking data points.

What is sampling?

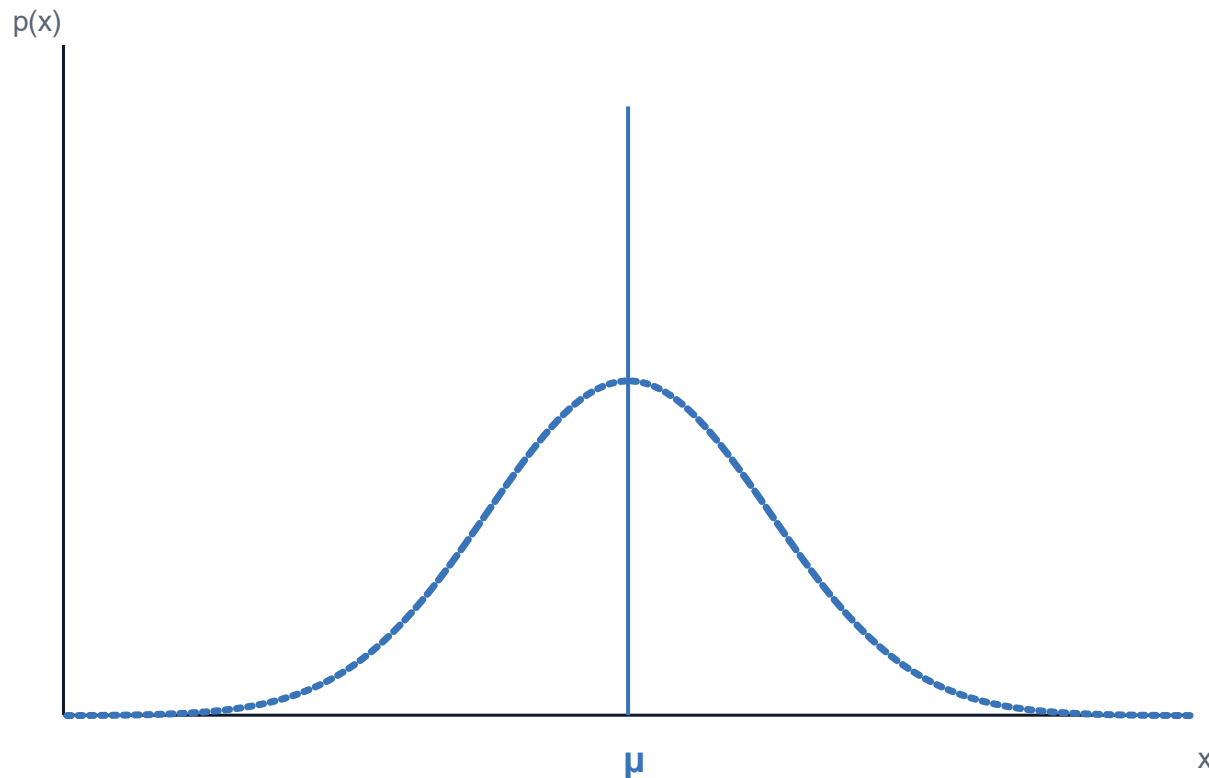
Drawing a random outcome according to a probability distribution



High-density regions appear more often—not exclusively

The One-Dimensional Gaussian

A bell-shaped probability density over one real number



$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

$$x \sim \mathcal{N}(\mu, \sigma^2)$$

μ = location

moves the center

σ = scale

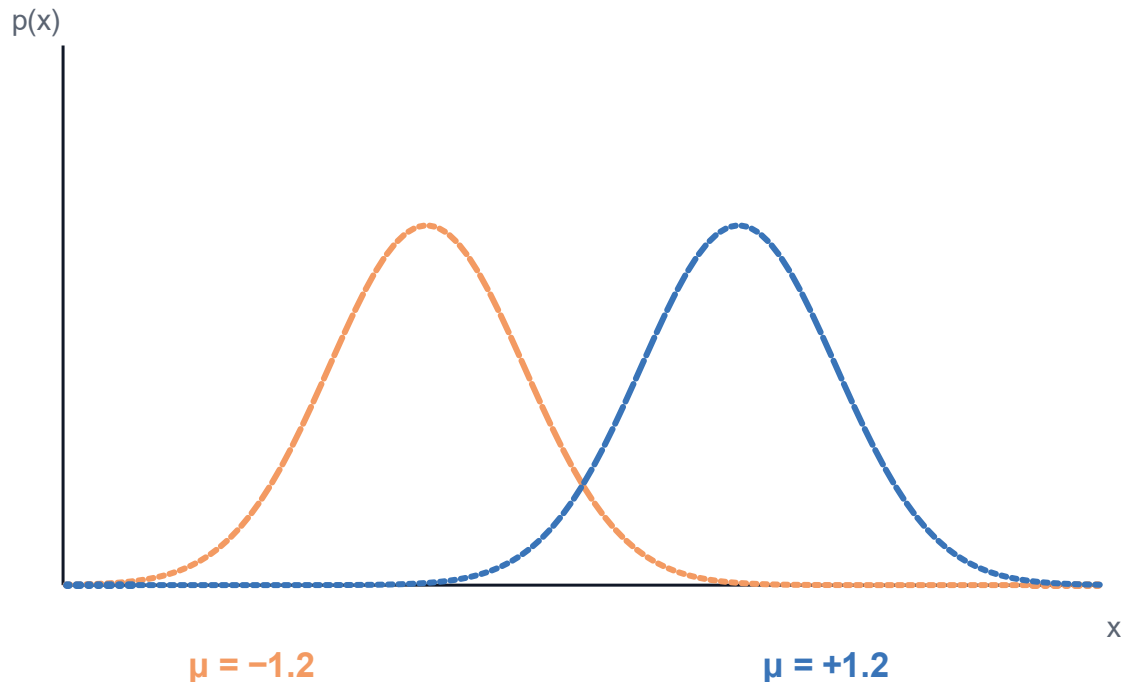
controls the spread

$\sigma > 0$, and the total area under the curve is 1

How μ and σ Change the Curve

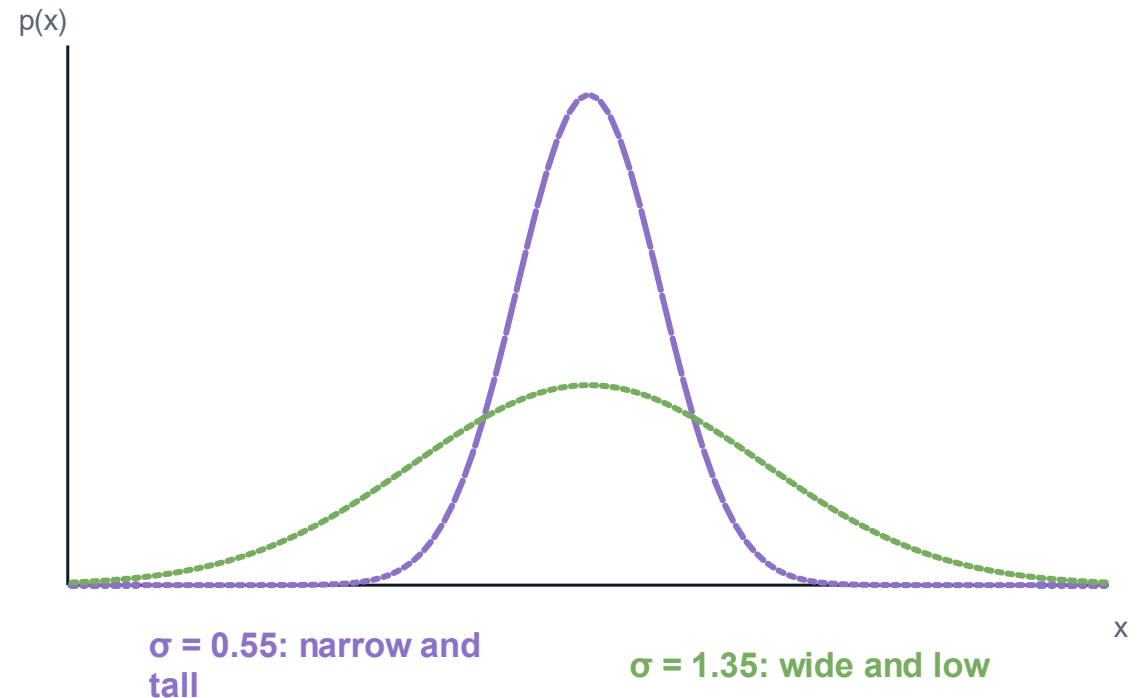
Mean shifts the center, standard deviation changes height and width.

Changing μ (same $\sigma = 0.75$)



Changing μ does not change shape

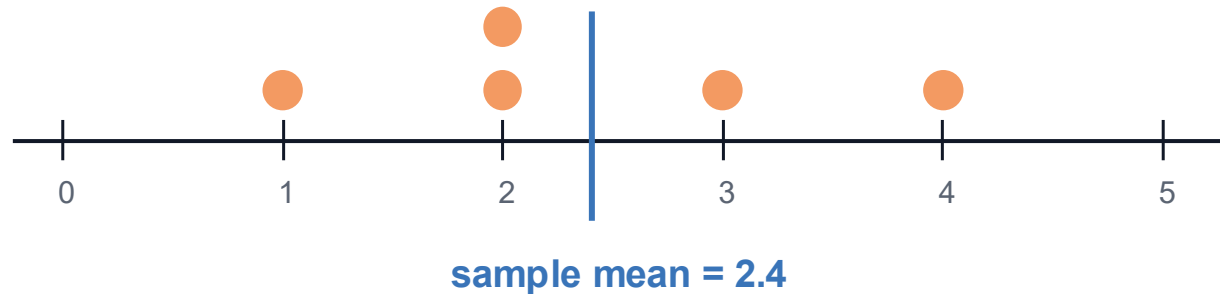
Changing σ (same $\mu = 0$)



Larger σ means greater uncertainty

Estimating a 1D Gaussian from Samples

Example: $x = \{1, 2, 2, 3, 4\}$



Mean $\hat{\mu} = \frac{1}{n} \sum_i x_i = 12/5 = 2.4$

Variance $\hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \hat{\mu})^2 = 1.04$

Fitted model

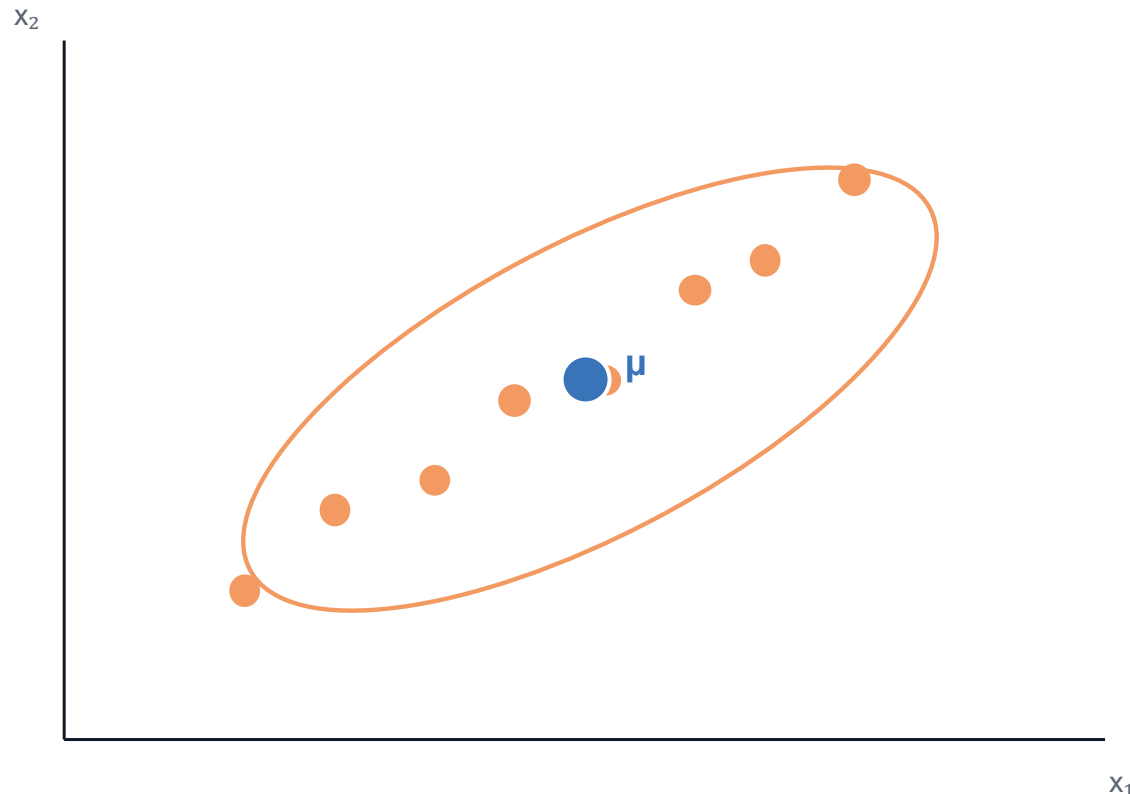
$$x \sim \mathcal{N}(2.4, 1.04)$$

The model stores two parameters,
not the five training values

Maximum-likelihood variance uses n ; an unbiased sample estimate uses $n-1$.

The Two-Dimensional Gaussian

A probability density over vectors $\mathbf{x} = [x_1, x_2]^\top$



$$p(\mathbf{x}) = \frac{1}{2\pi |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right)$$

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$$

$$\boldsymbol{\mu} = [\mu_1, \mu_2]^\top$$

center

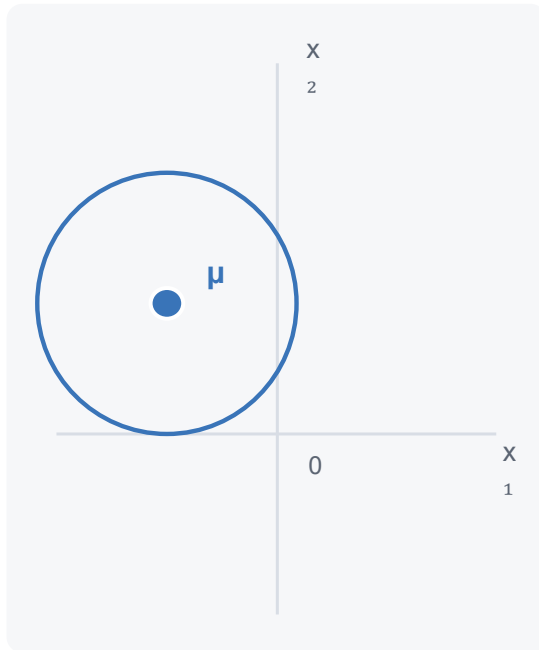
$$\Sigma = \text{covariance}$$

size, shape, tilt

Equal-density contours are ellipses.

How μ and Σ Change a 2D Gaussian

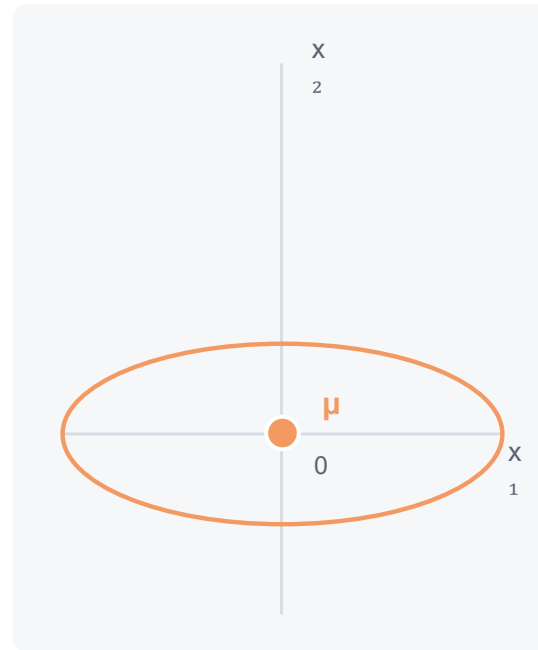
Mean shift



$$\mu = [-1, 1]^T$$

$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

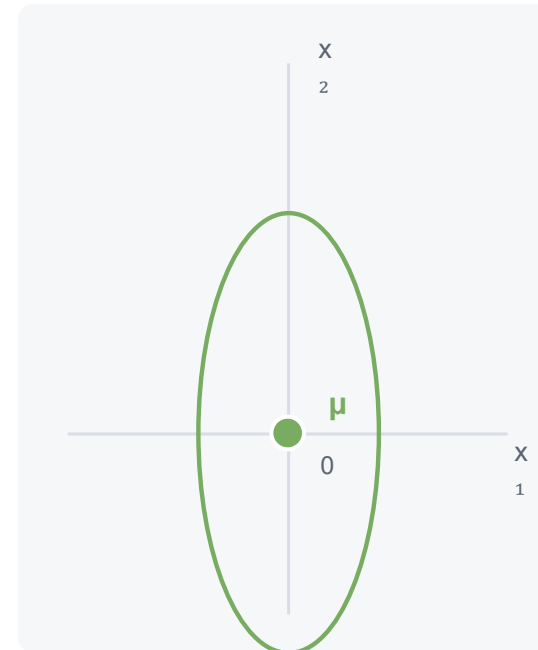
Horizontal spread



$$\mu = [0, 0]^T$$

$$\Sigma = \begin{bmatrix} 4 & 0 \\ 0 & 1 \end{bmatrix}$$

Vertical spread



$$\mu = [0, 0]^T$$

$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}$$

Positive covariance



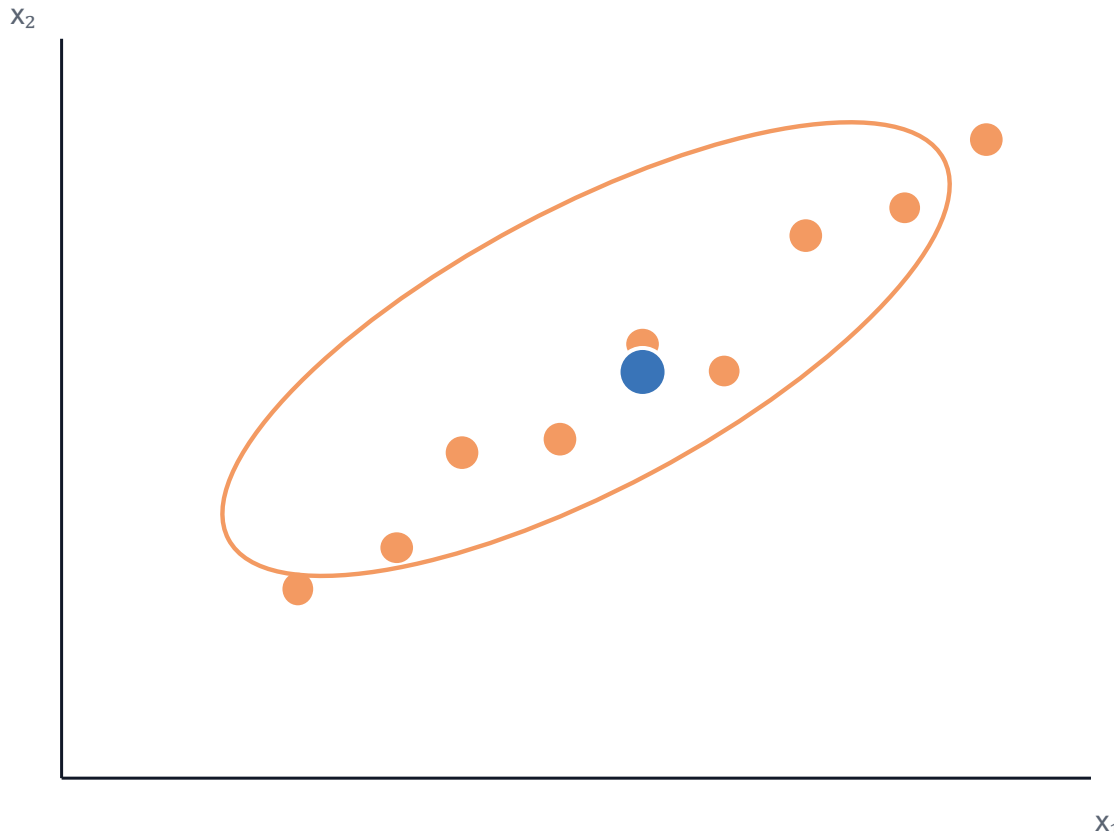
$$\mu = [0, 0]^T$$

$$\Sigma = \begin{bmatrix} 2 & 1.4 \\ 1.4 & 2 \end{bmatrix}$$

μ sets the center. Σ sets the horizontal and vertical spread, correlation, and rotation.

Estimating μ and Σ from 2D Samples

Compute a center, then average outer products of deviations



Mean $\hat{\mu} = \frac{1}{n} \sum_i x_i$

Covariance $\hat{\Sigma} = \frac{1}{n} \sum_i (x_i - \hat{\mu})(x_i - \hat{\mu})^T$

Example estimate

$$\hat{\mu} \approx [0.3, 3.3]^T$$

$$\hat{\Sigma} \approx \begin{bmatrix} 1.82 & 1.41 \\ 1.41 & 1.14 \end{bmatrix}$$

Positive off-diagonal values explain the upward tilt

How a Computer Samples a Gaussian

Uniform random bits $\rightarrow$ standard normal $\rightarrow$ desired mean and covariance

1D: Box-Muller + affine transform

1 Draw $u_1, u_2 \sim \text{Uniform}(0,1)$

2 $z = \sqrt{-2 \ln u_1} \cos(2\pi u_2)$

3 $z \sim \mathcal{N}(0,1)$

4 $x = \mu + \sigma z \Rightarrow x \sim \mathcal{N}(\mu, \sigma^2)$

Example: $\mu = 5, \sigma = 2, z = -0.4 \rightarrow x = 4.2$

2D: Cholesky transform

1 Draw $z = [z_1, z_2]^T$ with $z_1, z_2 \sim \mathcal{N}(0,1)$

2 Factor the covariance: $\Sigma = LL^T$

3 Return $x = \mu + Lz \Rightarrow x \sim \mathcal{N}(\mu, \Sigma)$

In practice, libraries implement these steps for us:

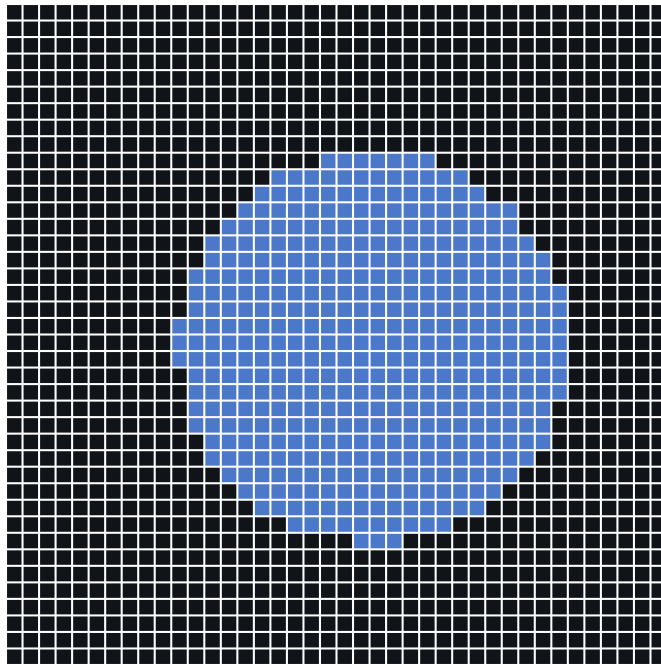
```
import numpy as np
```

```
rng = np.random.default_rng(seed=42)  
rng.multivariate_normal(mean= $\mu$ , cov= $\Sigma$ )
```

Two Ways to Generate the Same Circle

The output can look identical even when the model represents it differently

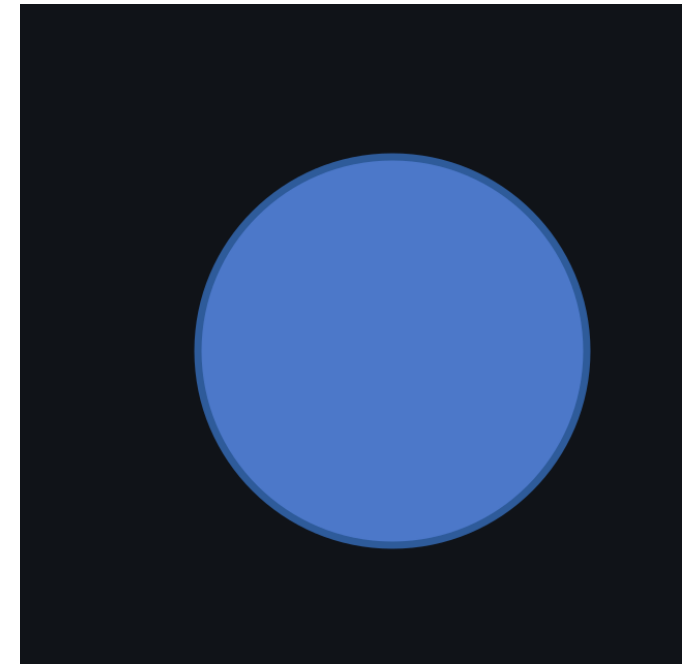
Autoregressive image model



$\mathbf{x} = (\text{patch}_1, \text{patch}_2, \dots, \text{patch}_{1600})$

Predict one patch after another

Latent variable model



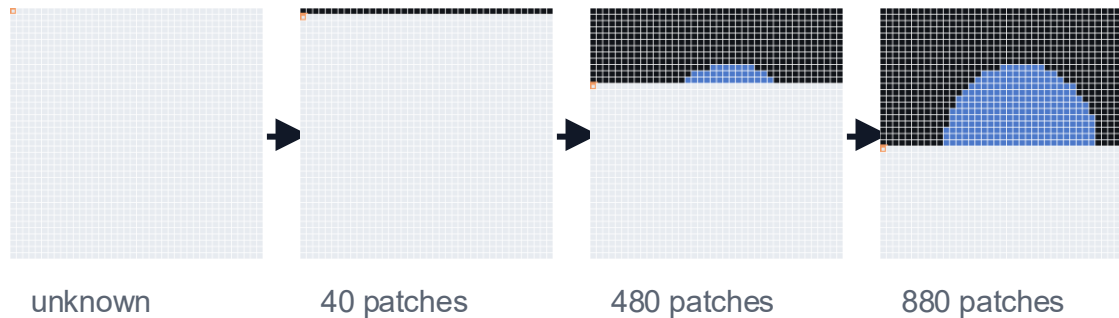
$\mathbf{z} = (\text{center}, \text{radius}, \text{color})$

Generate the whole image from $\mathbf{z}$

Autoregressive Generation

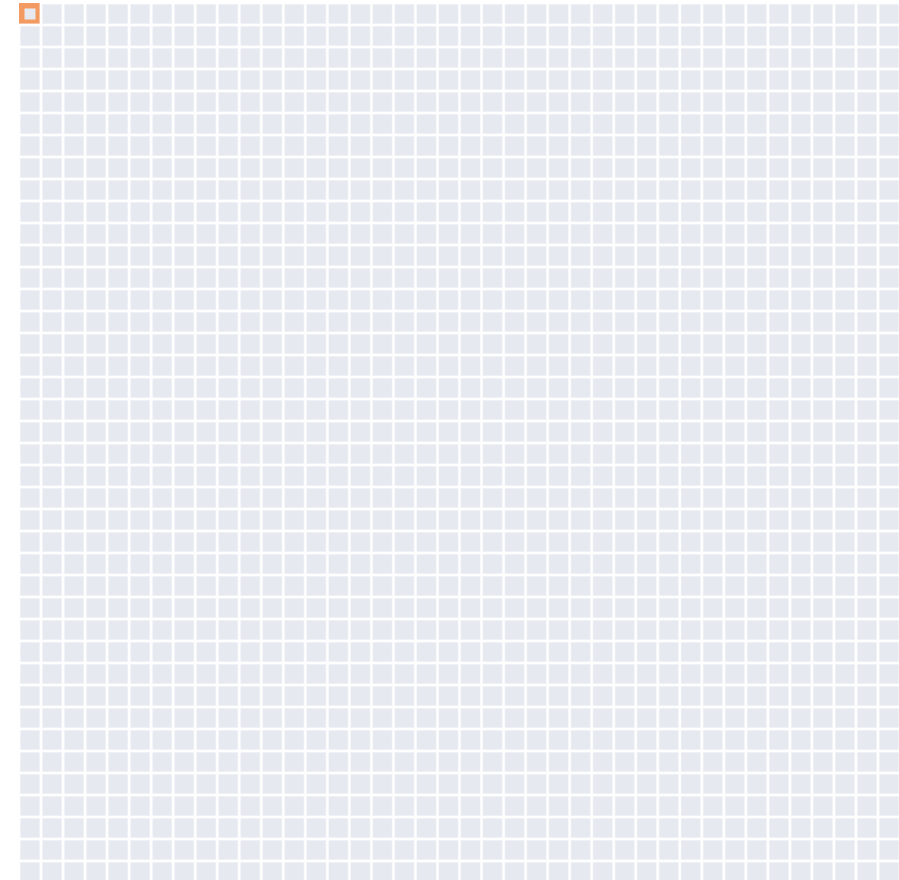
The model scans the image and predicts each patch from the patches already generated

$$p(\mathbf{x}) = \prod_i p(\mathbf{x}_i \mid \mathbf{x}_1, \dots, \mathbf{x}_{i-1})$$



At every step, the model chooses a color for the next patch

Early patches are mostly black background. Blue patches appear when the scan reaches the circle.

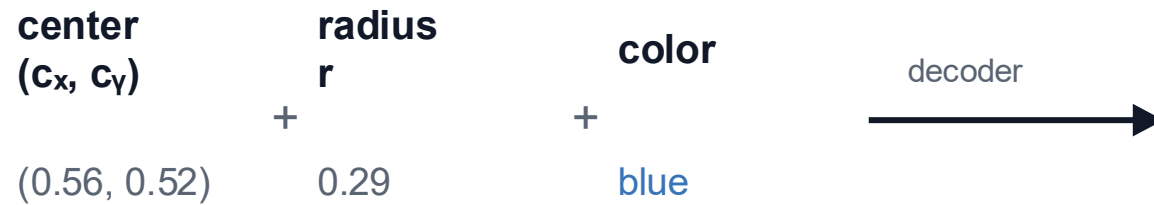


Animated 40 × 40 raster scan

Four consecutive patches appear per animation frame

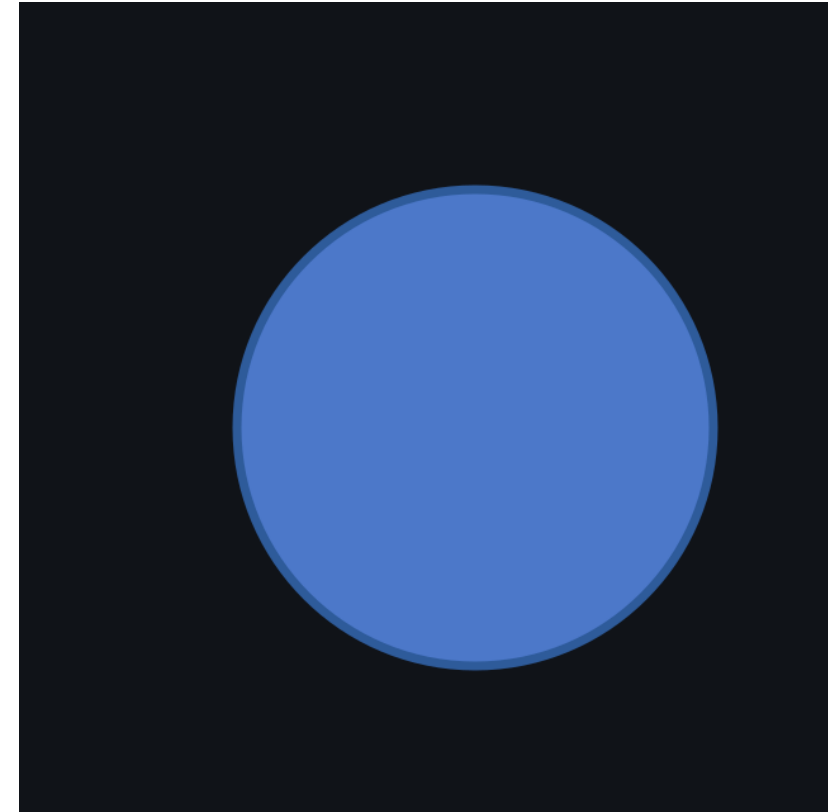
Latent Variable Generation

$$\mathbf{z} = (\mathbf{c}_x, \mathbf{c}_y, r, \text{color})$$



$$p(\mathbf{x}) = \int p(\mathbf{x} | \mathbf{z})p(\mathbf{z}) d\mathbf{z}$$

Sample $\mathbf{z}$ once. The decoder produces all pixels together.



Changing $\mathbf{z}$ changes the rendered circle

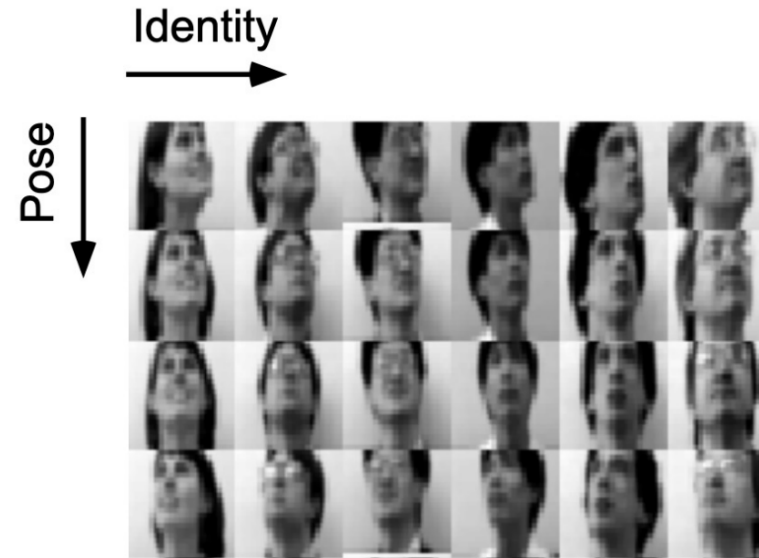
position moves it radius resizes it color recolors it

Probabilistic Modeling

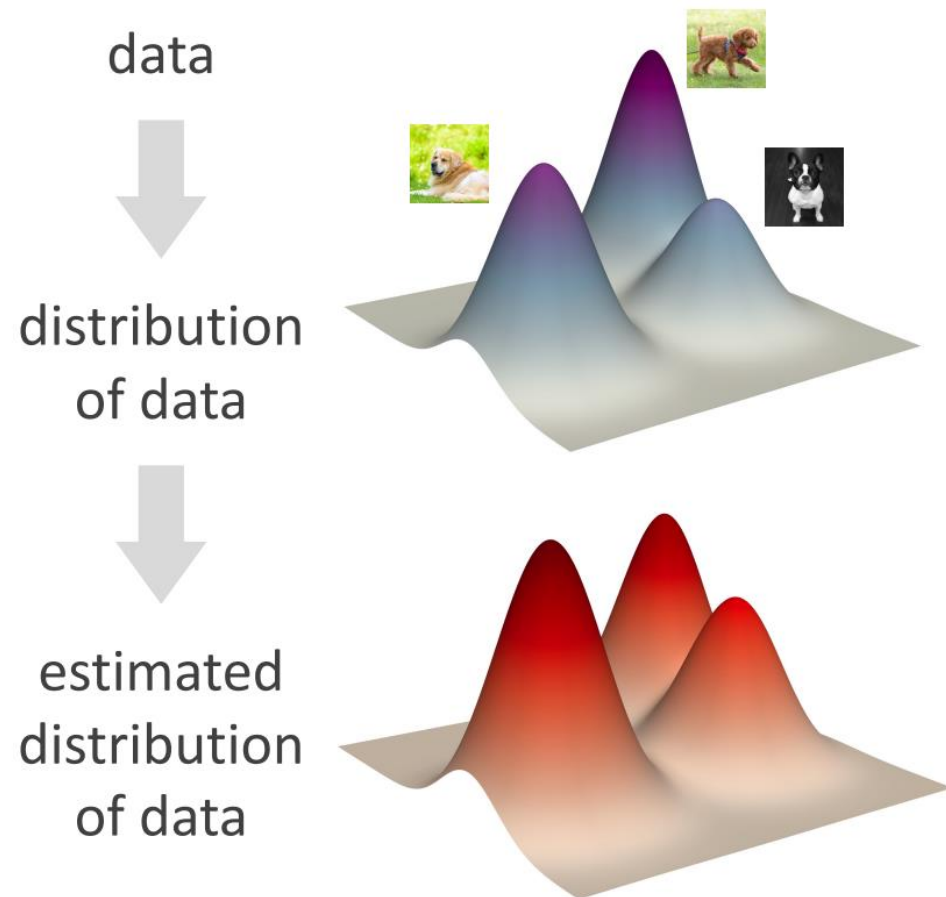
- Where does probability come from?
- Assuming underlying **distributions of data generation process**

example:

- latent factors z (pose, lighting, scale, ...)
 - z has simple distributions
 - observations x are rendered by a “world model” that’s a function on z
 - observations x have complex distributions
-
- Probability is part of the modeling.



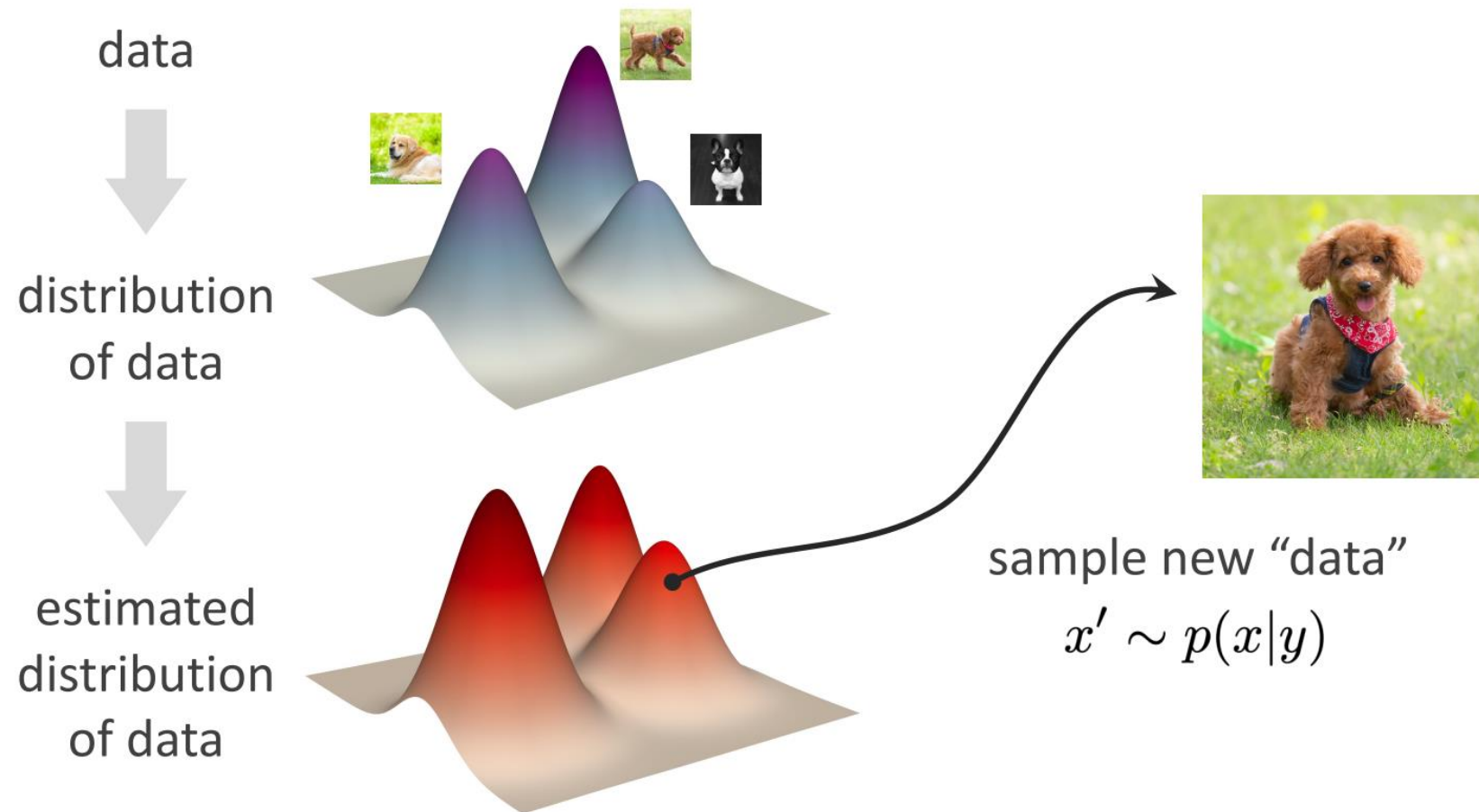
Probabilistic Modeling



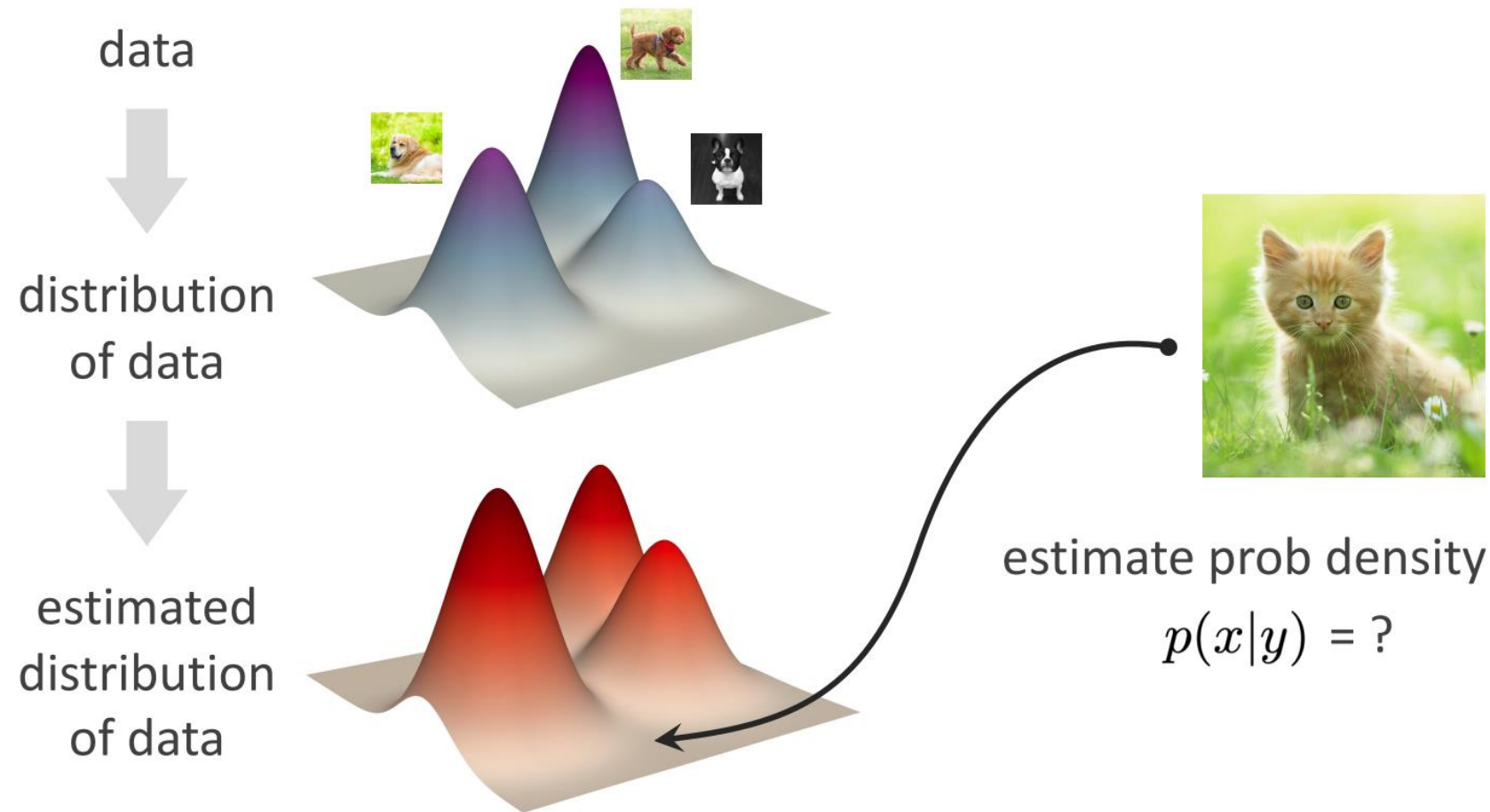
- Optimize a loss function

$$\mathcal{L}(\text{distribution of data}, \text{estimated distribution of data})$$

Probabilistic Modeling



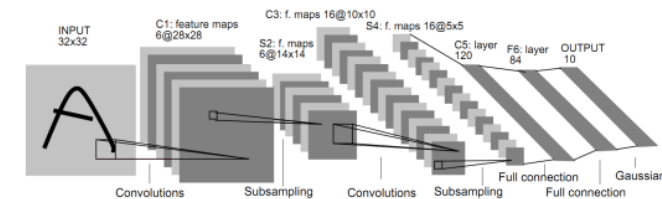
Probabilistic Modeling



Generative Models

- Deep learning is **representation learning**
- Learning to represent data instances
 - map data to feature: $x \rightarrow f(x)$
 - minimize loss w/ target: $\mathcal{L}(y, f(x))$

$$x \longrightarrow f(x)$$



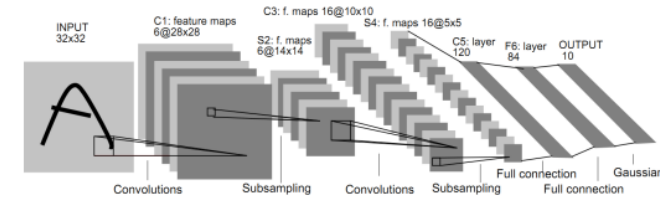
Generative Models

- Deep learning is **representation learning**

$$x \longrightarrow f(x)$$

- Learning to represent data instances

- map data to feature: $x \rightarrow f(x)$
- minimize loss w/ target: $\mathcal{L}(y, f(x))$



$$g(\pi) \longleftarrow \pi$$

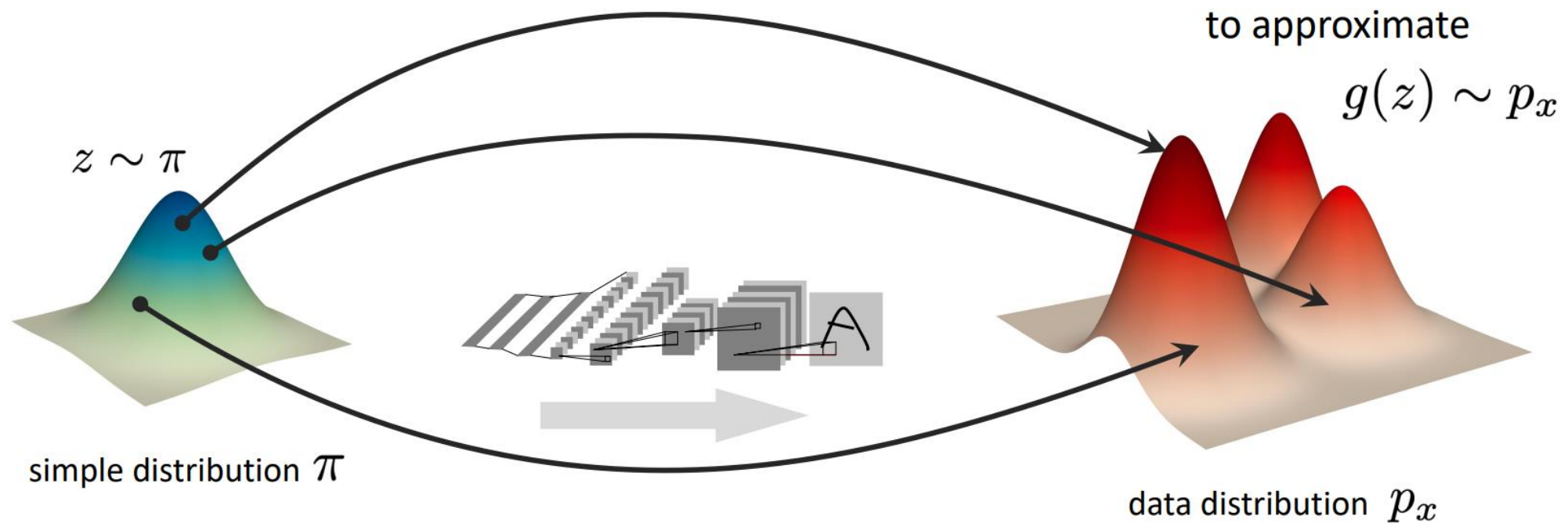
- Learning to **represent probability distributions**

- map a simple distribution (Gaussian/uniform) to a complex one: $\pi \rightarrow g(\pi)$
- minimize loss w/ data distribution: $\mathcal{L}(p_x, g(\pi))$

- Often perform both together

Learning to Represent Probability Distributions

- From simple to complex distributions



Generative Models may involve:

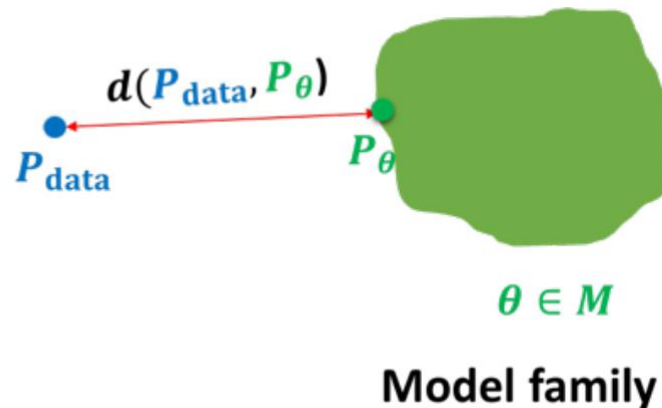
- **Formulation:**
 - formulate a problem as probabilistic modeling
 - decompose complex distributions into simple and tractable ones
- **Representation:**
 - deep neural networks to represent data and their distributions
- **Objective function:**
 - to measure how good the predicted distribution is
- **Optimization:**
 - optimize the networks and/or the decomposition
- **Inference:**
 - sampler: to produce new samples
 - probability density estimator (optional)

Learning a generative model

- We are given a training set of examples, e.g., images of dogs



$$x_i \sim P_{\text{data}} \\ i = 1, 2, \dots, n$$



- We want to learn a probability distribution $p(x)$ over images x such that
 - Generation:** If we sample $x_{\text{new}} \sim p(x)$, x_{new} should look like a dog (sampling)
 - Density estimation:** $p(x)$ should be high if x looks like a dog, and low otherwise (anomaly detection)
 - Unsupervised representation learning:** We should be able to learn what these images have in common, e.g., ears, tail, etc. (features)
- First question: how to represent $p(x)$

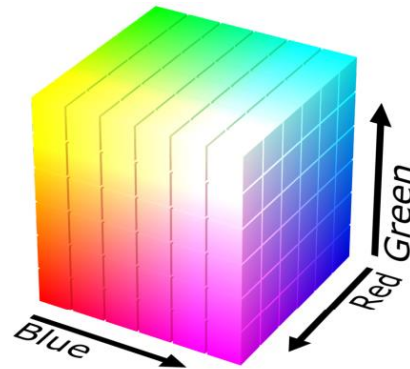
Basic discrete distributions

- Bernoulli distribution: (biased) coin flip
 - $D = \{Heads, Tails\}$
 - Specify $P(X = Heads) = p$. Then $P(X = Tails) = 1 - p$.
 - Write: $X \sim Ber(p)$
 - Sampling: flip a (biased) coin
- Categorical distribution: (biased) m-sided dice
 - $D = \{1, \dots, m\}$
 - Specify $P(Y = i) = p_i$, such that $\sum p_i = 1$.
 - Write: $Y \sim Cat(p_1, \dots, p_m)$
 - Sampling: roll a (biased) dice

Example of joint distribution

Modeling a single pixel's color. Three discrete random variables:

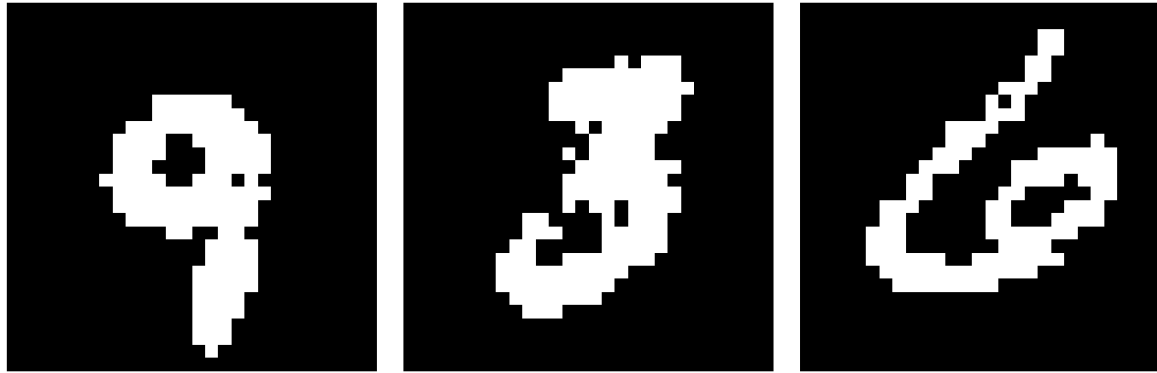
- Red Channel R. $\text{Val}(R) = \{0, \dots, 255\}$
- Green Channel G. $\text{Val}(G) = \{0, \dots, 255\}$
- Blue Channel B. $\text{Val}(B) = \{0, \dots, 255\}$



- Sampling from the joint distribution $(r, g, b) \sim p(R, G, B)$ randomly generates a color for the pixel.
- How many parameters do we need to specify the joint distribution $p(R = r, G = g, B = b)$?

$$256 * 256 * 256 - 1$$

Example of joint distribution



- Suppose $X_1, \dots, X_n$ are binary (Bernoulli) random variables, i.e., $\text{Val}(X_i) = \{0, 1\} = \{\text{Black}, \text{White}\}$.
- How many possible images (states)?

$$\underbrace{2 \times 2 \times \dots \times 2}_{n \text{ times}} = 2^n$$

- Sampling from $p(X_1, \dots, X_n)$ generates an image
- How many parameters to specify the joint distribution $p(X_1, \dots, X_n)$ over n binary pixels?

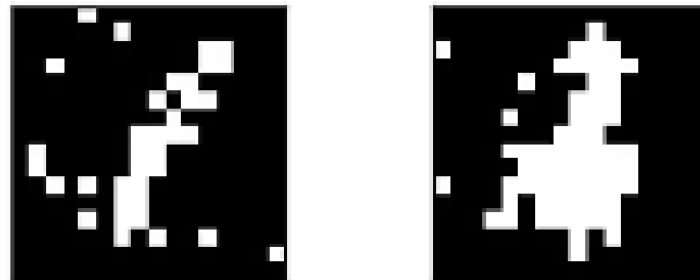
$$2^n - 1$$

Structure via independence

- If $\{X_1, \dots, X_n\}$ are independent, then

$$p(x_1, \dots, x_n) = p(x_1)p(x_2) \cdots p(x_n)$$

- How many possible states? 2^n
- How many parameters to specify the joint distribution $p(X_1, \dots, X_n)$?
 - How many to specify the marginal distribution $p(X_1)$? **1**
- 2^n **entries can be described by just n numbers** (if $|\text{Val}(X_i)| = 2$)!
- Independence assumption is too strong. Model not likely to be useful
 - For example, each pixel chosen independently when we sample from it.



Two important rules

Chain rule Let $S_1, \dots, S_n$ be events, $p(S_i) > 0$.

$$p(S_1 \cap S_2 \cap \dots \cap S_n) = p(S_1)p(S_2 | S_1) \cdots p(S_n | S_1 \cap \dots \cap S_{n-1})$$

Bayes' rule Let S_1, S_2 be events, $p(S_1) > 0$ and $p(S_2) > 0$.

$$p(S_1 | S_2) = \frac{p(S_1 \cap S_2)}{p(S_2)} = \frac{p(S_2 | S_1)p(S_1)}{p(S_2)}$$

Structure via conditional independence

- Using Chain Rule

$$p(x_1, \dots, x_n) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \cdots p(x_n \mid x_1, \dots, x_{n-1})$$

- How many parameters? $1 + 2 + \dots + 2^{n-1} = 2^n - 1$
 - $p(x_1)$ requires 1 parameter
 - $p(x_2 \mid x_1 = 0)$ requires 1 parameter, $p(x_2 \mid x_1 = 1)$ requires 1 parameter
Total 2 parameters.
 - ...
- $2^n - 1$ is still exponential, chain rule does not buy us anything.
- Now suppose $x_{i+1} \perp x_1, \dots, x_{i-1} \mid x_i$, then

$$\begin{aligned} p(x_1, \dots, x_n) &= p(x_1)p(x_2 \mid x_1)p(x_3 \mid \cancel{x_1}, x_2) \cdots p(x_n \mid \cancel{x_1, \dots, x_{i-1}}, x_{n-1}) \\ &= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_2) \cdots p(x_n \mid x_{n-1}) \end{aligned}$$

- How many parameters? $2n - 1$. Exponential reduction!

Bayes Network: General Idea

- Use conditional parameterization (instead of joint parameterization)
- For each random variable X_i , specify $p(X_i|X_{A_i})$ for set X_{A_i} of random variables
- Then get joint parametrization as:

$$p(x_1, \dots, x_n) = \prod_i p(x_i | \mathbf{x}_{A_i})$$

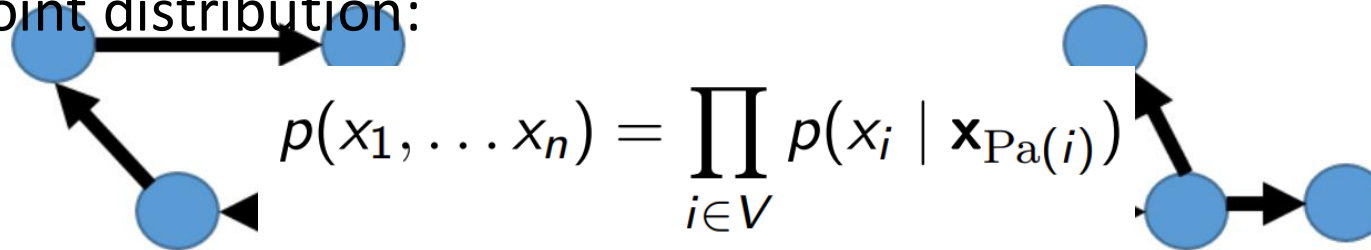
- Need to guarantee it is a **legal** probability distribution. It has to correspond to a chain rule factorization, with factors simplified due to assumed conditional independencies.

Bayesian networks

Bayesian networks

- A **Bayesian network** is specified by a **directed acyclic graph** (DAG) $G = (V, E)$ with:
 - One node $i \in V$ for each random variable X_i
 - One conditional probability distribution (CPD) per node, $p(X_i | X_{Pa(i)})$, specifying the variable's probability conditioned on its parents' values

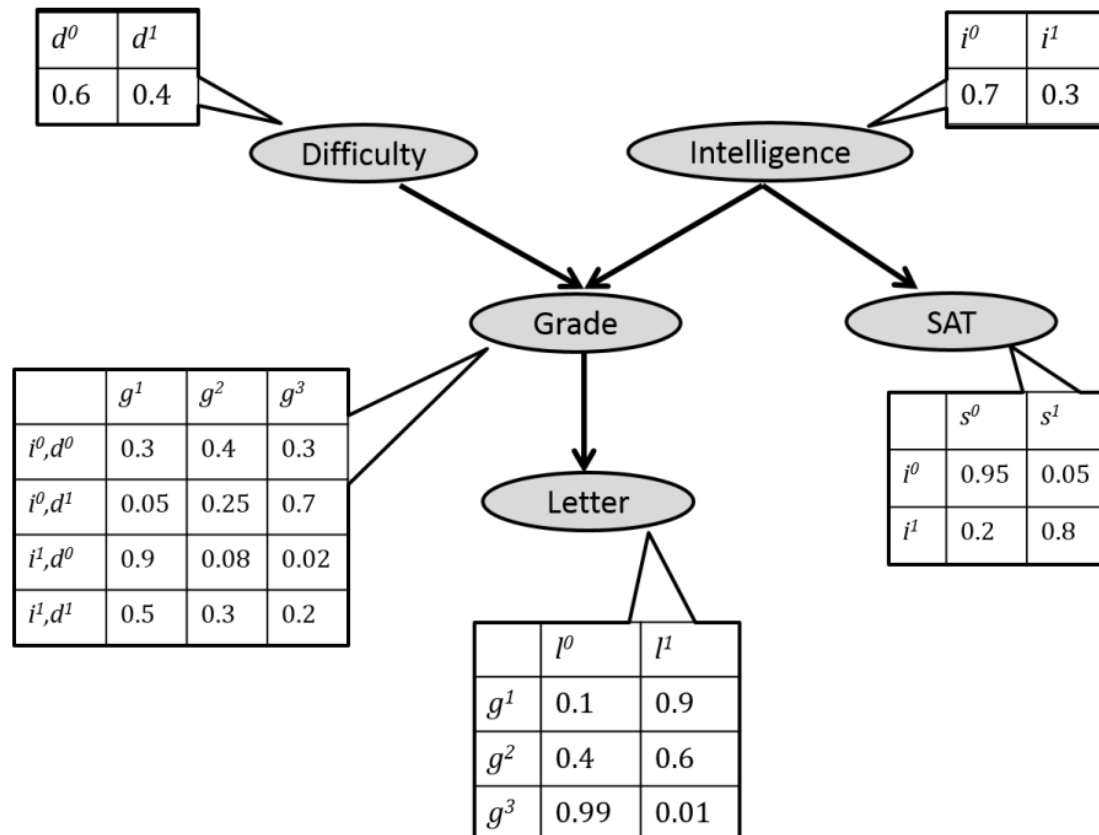
- Graph $G = (V, E)$ is called the structure of the Bayesian Network
- Defines a joint distribution:

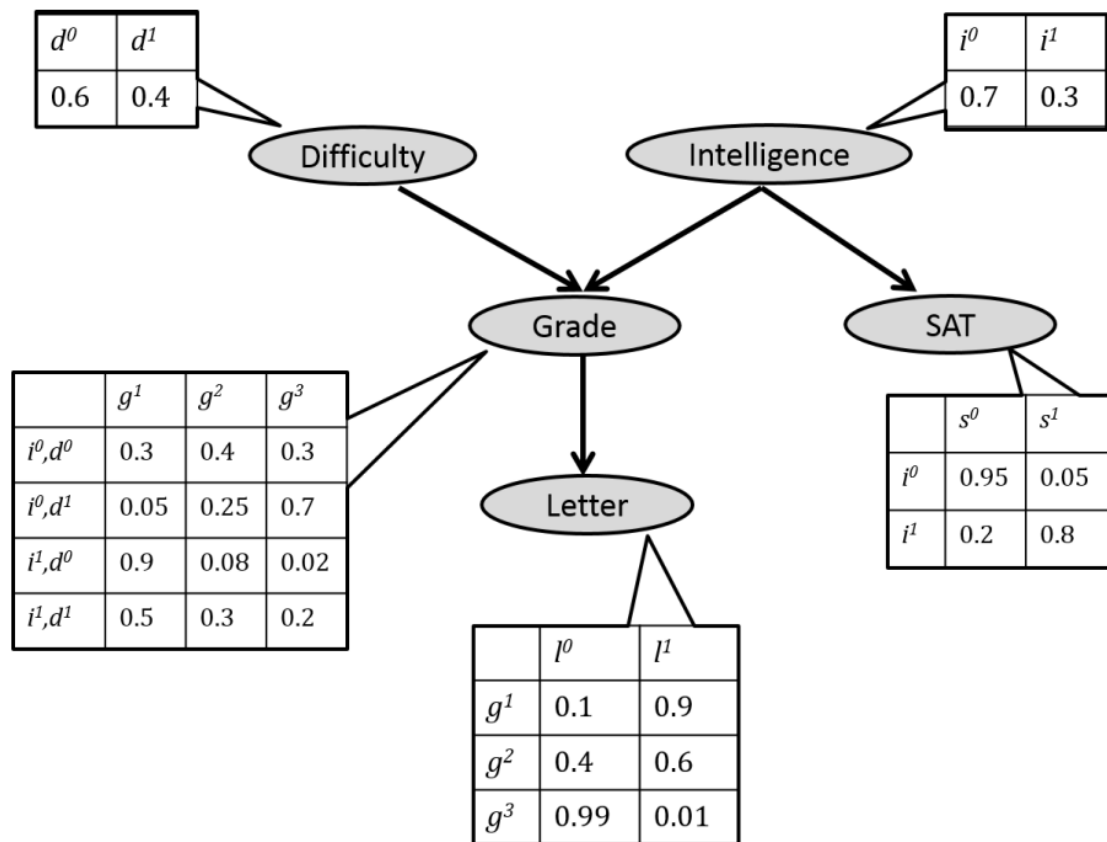


- Claim: $p(X_1, \dots, X_n)$ is a valid probability distribution because of ordering implied by DAG
- Economical representation: exponential in $|Pa(i)|$, not $|V|$

Example

Consider the following Bayesian network:



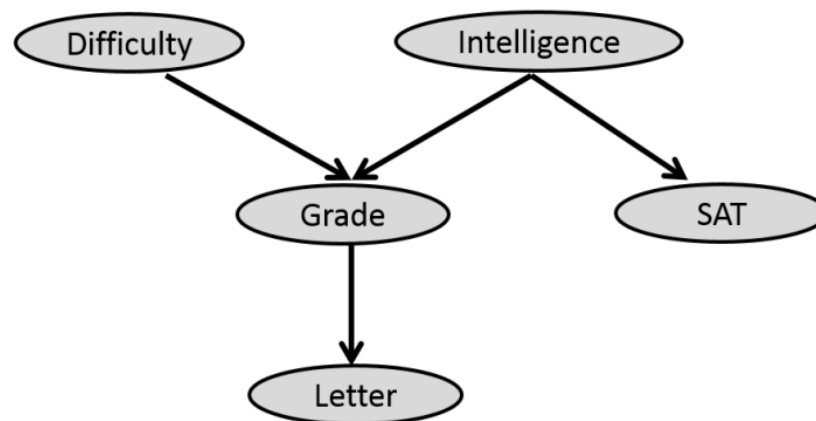


What is its joint distribution?

$$p(x_1, \dots, x_n) = \prod_{i \in V} p(x_i \mid \mathbf{x}_{\text{Pa}(i)})$$

$$p(d, i, g, s, l) = p(d)p(i)p(g \mid i, d)p(s \mid i)p(l \mid g)$$

Bayesian network structure implies conditional independencies



- The joint distribution corresponding to the above Bayesian Network factors as

$$p(d, i, g, s, l) = p(d)p(i)p(g | i, d)p(s | i)p(l | g)$$

- However, by the chain rule, any distribution can be written as

$$p(d, i, g, s, l) = p(d)p(i | d)p(g | i, d)p(s | i, d, g)p(l | g, d, i, s)$$

- Thus, we are assuming the following additional independencies:

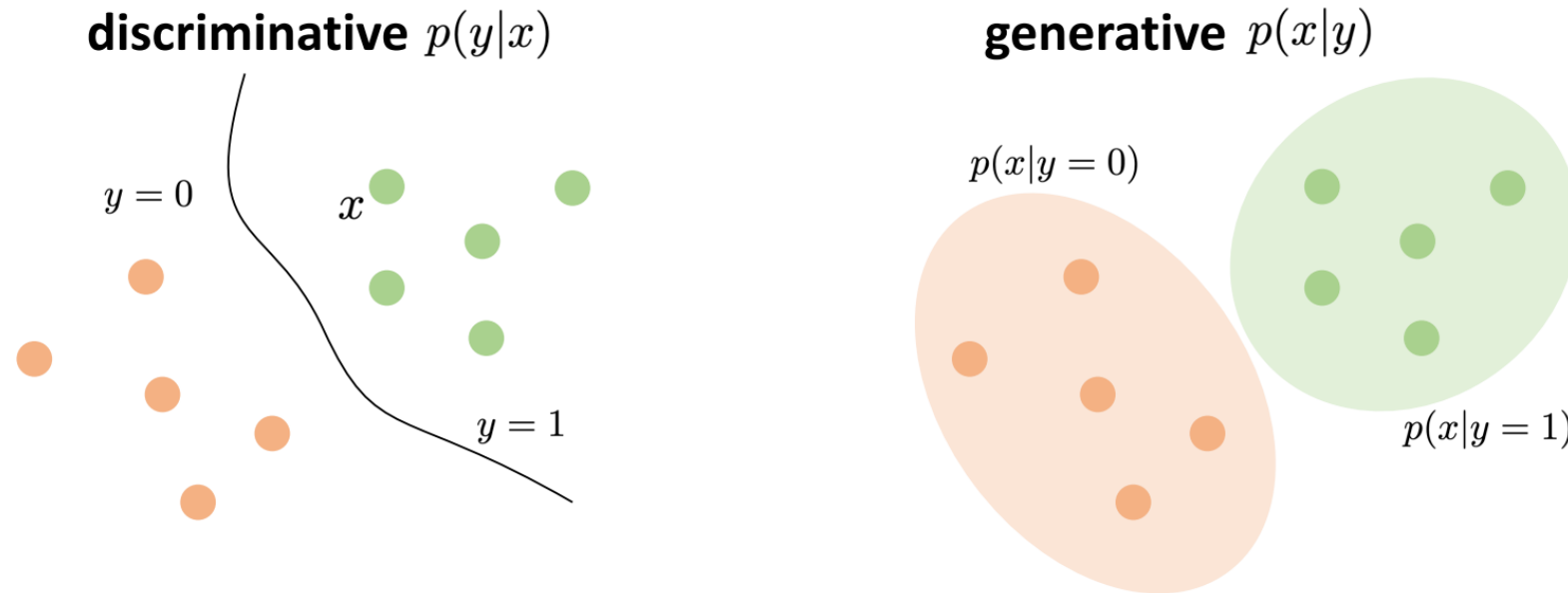
$$D \perp I, \quad S \perp \{D, G\} | I, \quad L \perp \{I, D, S\} | G.$$

Bayesian networks Summary

- **Bayesian networks** given by (G, P) , where P is specified as a set of local **conditional probability distributions** associated with G 's nodes
- Efficient representation using a graph-based data structure
- Computing the probability of any assignment is obtained by multiplying CPDs
- Can sample from the joint by sampling from the CPDs according to the DAG ordering
- Can identify some conditional independence properties by looking at graph properties
- In this class, graphical models will be simple (e.g., only 2 or 3 random vectors)
- Next: generative vs. discriminative; functional parameterizations

Naive Bayes for single label prediction

Example: naive Bayes for classification



- Generative models can be discriminative: Bayes' rule

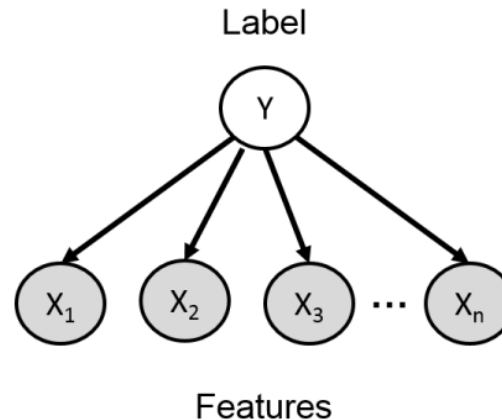
$$\underbrace{p(y|x)}_{\text{discriminative}} = \underbrace{p(x|y)}_{\text{generative}} \frac{p(y)}{p(x)}$$

assuming known prior (pointing to $p(y)$)

constant for given x (pointing to $p(x)$)

Example: naive Bayes for classification

- Classify e-mails as spam ($Y = 1$) or not spam ($Y = 0$)
 - Let $1:n$ index the words in our vocabulary (e.g., English)
 - $X_i = 1$ if word i appears in an e-mail, and 0 otherwise
 - E-mails are drawn according to some distribution $p(Y, X_1, \dots, X_n)$
- Words are conditionally independent given Y :



- Then:

$$p(y, x_1, \dots, x_n) = p(y) \prod_{i=1}^n p(x_i | y)$$

Example: naive Bayes for classification

- Classify e-mails as spam ($Y = 1$) or not spam ($Y = 0$)
 - Let $1:n$ index the words in our vocabulary (e.g., English)
 - $X_i = 1$ if word i appears in an e-mail, and 0 otherwise
 - E-mails are drawn according to some distribution $p(Y, X_1, \dots, X_n)$
- Suppose that the words are conditionally independent given Y . Then

$$p(y, x_1, \dots, x_n) = p(y) \prod_{i=1}^n p(x_i | y)$$

- Estimate parameters from training data. Predict with Bayes rule:

$$p(Y = 1 | x_1, \dots, x_n) = \frac{p(Y = 1) \prod_{i=1}^n p(x_i | Y = 1)}{\sum_{y=\{0,1\}} p(Y = y) \prod_{i=1}^n p(x_i | Y = y)}$$

Example: naive Bayes for classification

Ten concrete emails represented by three binary word features.

ID	Class	Example email text	X ₁ free	X ₂ win	X ₃ meeting
S1	Spam	Free offer, win a prize today	1	1	0
S2	Spam	Win free tickets now	1	1	0
S3	Spam	Claim your free deal	1	0	0
S4	Spam	Win cash today	0	1	0
S5	Spam	Meeting bonus update	0	0	1
N1	Non-spam	Meeting agenda for Monday	0	0	1
N2	Non-spam	Team meeting tomorrow	0	0	1
N3	Non-spam	Project meeting notes	0	0	1
N4	Non-spam	Free meeting room available	1	0	1
N5	Non-spam	Lunch schedule for Friday	0	0	0

1 = word present 0 = word absent

$P(Y=1)=5/10=0.5$

$P(Y=0)=5/10=0.5$

Example: naive Bayes for classification

Each fraction counts emails within one class

Feature	Spam count	$p(X_i=1 \mid Y=1)$	Non-spam count	$p(X_i=1 \mid Y=0)$
X_1 : "free"	3 of 5	0.6	1 of 5	0.2
X_2 : "win"	3 of 5	0.6	0 of 5	0.0
X_3 : "meeting"	1 of 5	0.2	4 of 5	0.8

Absence probabilities

Spam $p(\text{free}=0)=0.4$ $p(\text{win}=0)=0.4$
 $p(\text{meeting}=0)=0.8$

Non-spam $p(\text{free}=0)=0.8$ $p(\text{win}=0)=1.0$
 $p(\text{meeting}=0)=0.2$

Example: naive Bayes for classification

New email: “Free meeting: win a prize”

Observed feature vector

$X_1 = 1$ free

$X_2 = 1$ win

$X_3 = 1$ meeting

$x = (1, 1, 1)$

For either class y : $\text{score}(y) = p(y, x) = p(y) p(X_1=1|y) p(X_2=1|y) p(X_3=1|y)$

Spam: $y = 1$

$$\begin{aligned}\text{score}(1) &= 0.5 \times 0.6 \times 0.6 \times 0.2 \\ &= 0.5 \times 0.36 \times 0.2 \\ &= 0.5 \times 0.072 \\ &= 0.036\end{aligned}$$

Non-spam: $y = 0$

$$\begin{aligned}\text{score}(0) &= 0.5 \times 0.2 \times 0 \times 0.8 \\ &= 0.5 \times 0 \times 0.8 \\ &= 0 \\ &\text{because } p(X_2=1 | Y=0)=0\end{aligned}$$

The two scores are joint probabilities. They are not normalized yet. $p(Y = 1 | x_1, \dots, x_n) = \frac{p(Y = 1) \prod_{i=1}^n p(x_i | Y = 1)}{\sum_{y \in \{0,1\}} p(Y = y) \prod_{i=1}^n p(x_i | Y = y)}$

Example: naive Bayes for classification

Step 1: calculate the denominator

$$p(x) = \text{score}(1) + \text{score}(0)$$

$$= 0.036 + 0$$

$$= 0.036$$

Check

$$p(Y=1|x) + p(Y=0|x)$$

$$= 1 + 0 = 1$$

Step 2: normalize each class score

$$p(Y=y | x) = \text{score}(y) / p(x)$$

$$p(Y=1 | x) = 0.036 / 0.036$$

$$= 1$$

$$p(Y=0 | x) = 0 / 0.036$$

$$= 0$$

Final prediction: SPAM ($Y = 1$)

Example: naive Bayes for classification

- Recap: two important rules

Chain rule Let $S_1, \dots, S_n$ be events, $p(S_i) > 0$.

$$p(S_1 \cap S_2 \cap \dots \cap S_n) = p(S_1)p(S_2 | S_1) \cdots p(S_n | S_1 \cap \dots \cap S_{n-1})$$

Bayes' rule Let S_1, S_2 be events, $p(S_1) > 0$ and $p(S_2) > 0$.

$$p(S_1 | S_2) = \frac{p(S_1 \cap S_2)}{p(S_2)} = \frac{p(S_2 | S_1)p(S_1)}{p(S_2)}$$

- Classify e-mails as spam ($Y = 1$) or not spam ($Y = 0$)
 - Let $1:n$ index the words in our vocabulary (e.g., English)
 - $X_i = 1$ if word i appears in an e-mail, and 0 otherwise
 - E-mails are drawn according to some distribution $p(Y, X_1, \dots, X_n)$
- Suppose that the words are conditionally independent given Y .

$$p(y, x_1, \dots, x_n) = p(y) \prod_{i=1}^n p(x_i | y)$$

- Estimate parameters from training data. Predict with Bayes rule:

$$p(Y = 1 | x_1, \dots, x_n) = \frac{p(Y = 1) \prod_{i=1}^n p(x_i | Y = 1)}{\sum_{y=\{0,1\}} p(Y = y) \prod_{i=1}^n p(x_i | Y = y)}$$

Example: naive Bayes for classification

- Classify e-mails as spam ($Y = 1$) or not spam ($Y = 0$)
 - Let $1:n$ index the words in our vocabulary (e.g., English)
 - $X_i = 1$ if word i appears in an e-mail, and 0 otherwise
 - E-mails are drawn according to some distribution $p(Y, X_1, \dots, X_n)$
- Suppose that the words are conditionally independent given Y . Then

$$p(y, x_1, \dots, x_n) = p(y) \prod_{i=1}^n p(x_i | y)$$

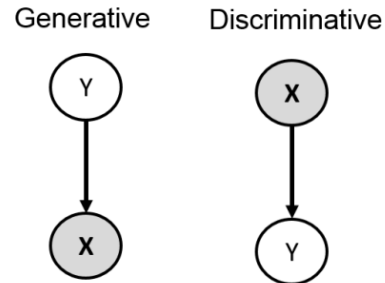
- Estimate parameters from training data. Predict with Bayes rule:

$$p(Y = 1 | x_1, \dots, x_n) = \frac{p(Y = 1) \prod_{i=1}^n p(x_i | Y = 1)}{\sum_{y=\{0,1\}} p(Y = y) \prod_{i=1}^n p(x_i | Y = y)}$$

- Are the independence assumptions made here reasonable?
- Philosophy: Nearly all probabilistic models are “wrong”, but many are nonetheless useful

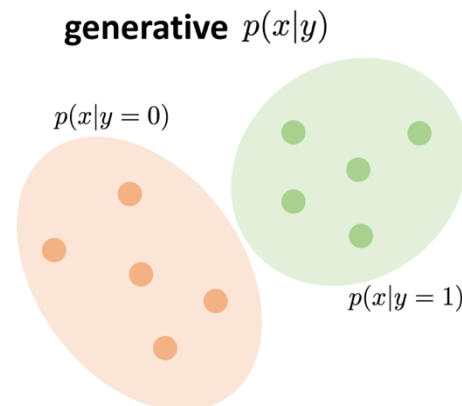
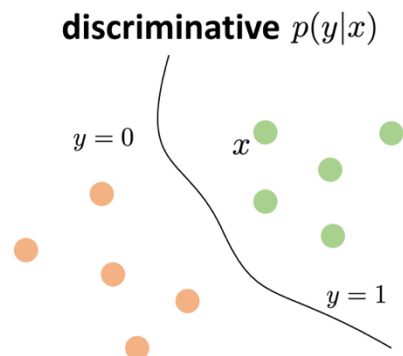
Discriminative vs. generative models

- Using chain rule $p(Y, \mathbf{X}) = p(\mathbf{X} | Y)p(Y) = p(Y | \mathbf{X})p(\mathbf{X})$
- Corresponding Bayesian networks:



- However, suppose all we need for prediction is $p(Y | \mathbf{X})$
- In the left model, we need to specify/learn both $p(Y)$ and $p(\mathbf{X} | Y)$, then compute $p(Y | \mathbf{X})$ via Bayes rule
- In the right model, it suffices to estimate just the **conditional distribution** $p(Y | \mathbf{X})$
 - We never need to model/learn/use $p(\mathbf{X})$!
 - Called a **discriminative** model because it is only useful for discriminating Y 's label when given $\mathbf{X}$

Discriminative vs. generative models



- Generative models can be discriminative: Bayes' rule

$$p(y|x) = p(x|y) \frac{p(y)}{p(x)}$$

discriminative generative

← assuming known prior
← constant for given x

- Can discriminative models be generative?

$$p(x|y) = p(y|x) \frac{p(x)}{p(y)}$$

generative discriminative

← still need to model prior distribution of x
← constant for given y

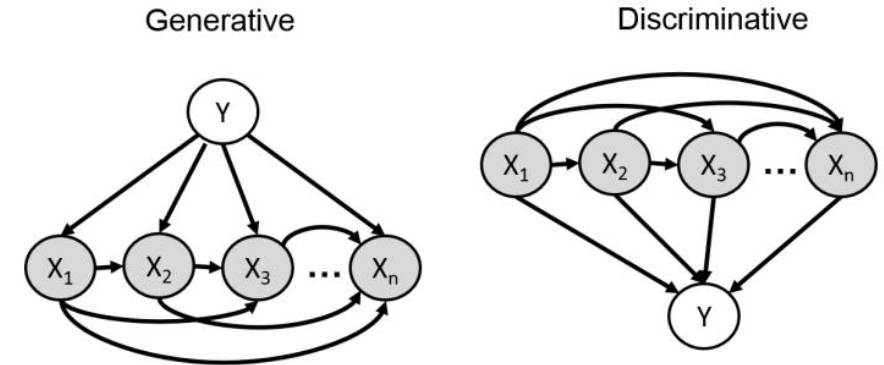
- The challenge is about representing and predicting distributions

Discriminative vs. generative models

- Since $\mathbf{X}$ is a random vector, chain rules will give

$$p(Y, \mathbf{X}) = p(Y)p(X_1 | Y)p(X_2 | Y, X_1) \cdots p(X_n | Y, X_1, \dots, X_{n-1})$$

$$p(Y, \mathbf{X}) = p(X_1)p(X_2 | X_1)p(X_3 | X_1, X_2) \cdots p(Y | X_1, \dots, X_{n-1}, X_n)$$

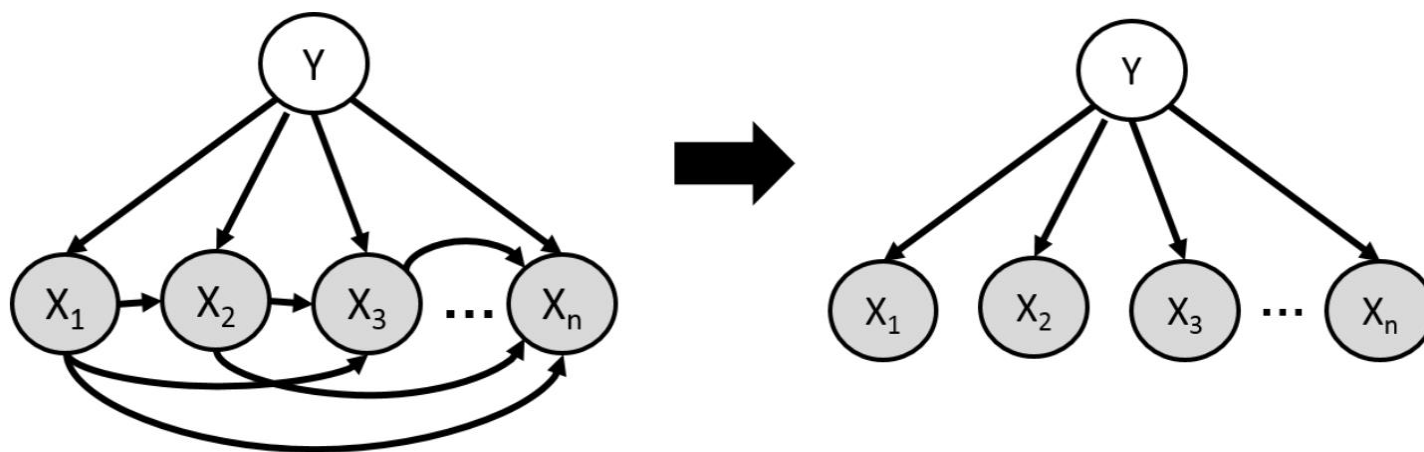


We must make the following choices:

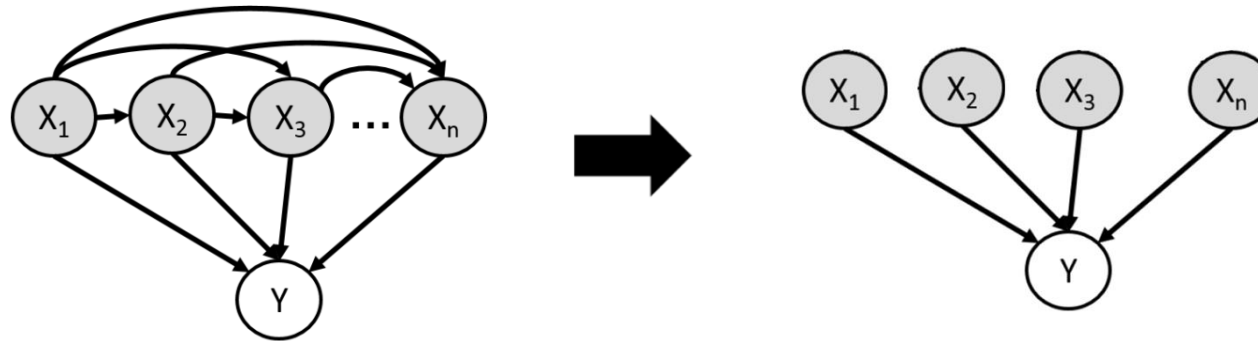
1. In the generative model, $p(Y)$ is simple, but how do we parameterize $p(X_i | \mathbf{X}_{pa(i)}, Y)$?
2. In the discriminative model, how do we parameterize $p(Y | \mathbf{X})$? Here we assume we don't care about modeling $p(\mathbf{X})$ because $\mathbf{X}$ is always given to us in a classification problem

Naive Bayes

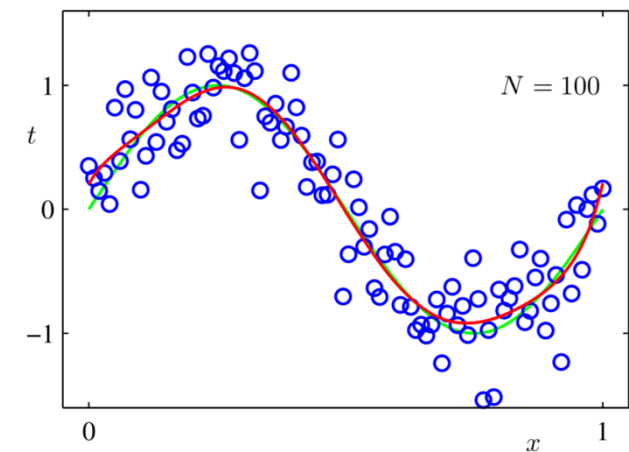
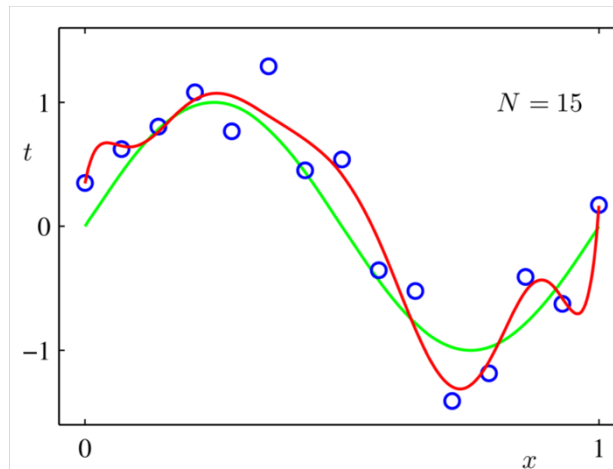
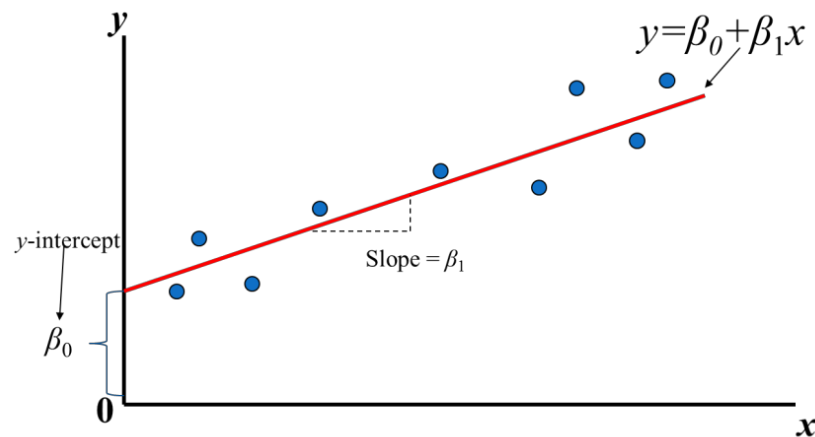
- For the generative model, assume that $X_i \perp \mathbf{X}_{-i} \mid Y$ (naive Bayes)



Logistic regression



- For the discriminative model, assume that
$$p(Y = 1 \mid \mathbf{x}; \alpha) = f(\mathbf{x}, \alpha)$$
- Not represented as a table anymore. It is a parameterized function of x (regression)



Logistic regression

- For the discriminative model, assume that

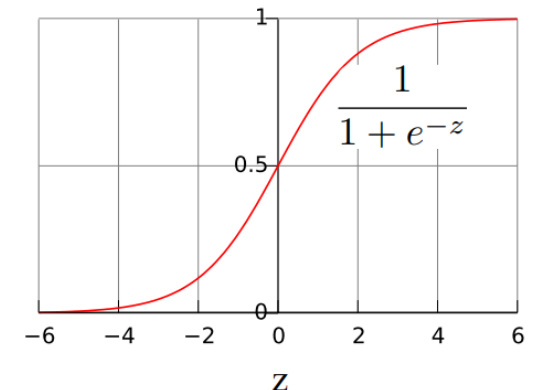
$$p(Y = 1 \mid \mathbf{x}; \boldsymbol{\alpha}) = f(\mathbf{x}, \boldsymbol{\alpha})$$

- Not represented as a table anymore. It is a parameterized function of $\mathbf{x}$ (regression)
 - Has to be between 0 and 1
 - Depend in some simple but reasonable way on $X_1, \dots, X_n$
 - Completely specified by a vector $\boldsymbol{\alpha}$ of $n+1$ parameters (**compact representation**)

Linear dependence: let $z(\boldsymbol{\alpha}, \mathbf{x}) = \alpha_0 + \sum_{i=1}^n \alpha_i x_i$.

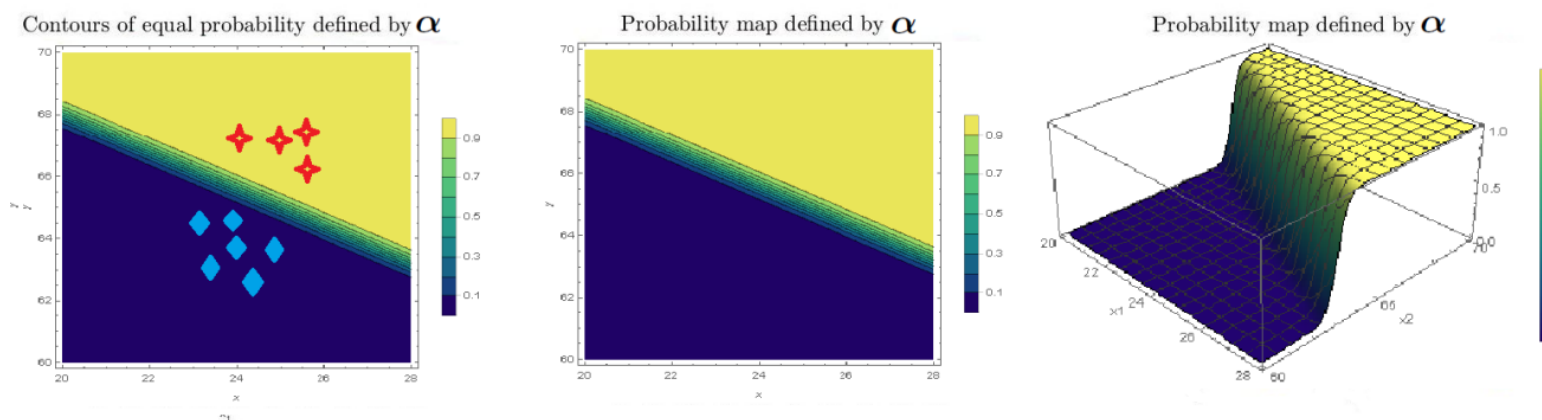
Then, $p(Y = 1 \mid \mathbf{x}; \boldsymbol{\alpha}) = \sigma(z(\boldsymbol{\alpha}, \mathbf{x}))$

where $\sigma(z) = 1/(1 + e^{-z})$ is called the **logistic function**:



Logistic regression

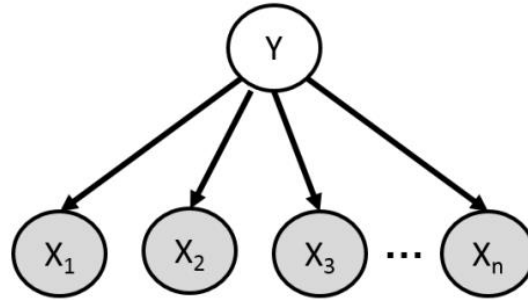
Linear dependence: let $z(\boldsymbol{\alpha}, \mathbf{x}) = \alpha_0 + \sum_{i=1}^n \alpha_i x_i$. Then, $p(Y = 1 \mid \mathbf{x}; \boldsymbol{\alpha}) = \sigma(z(\boldsymbol{\alpha}, \mathbf{x}))$, where $\sigma(z) = 1/(1 + e^{-z})$ is called the **logistic function**



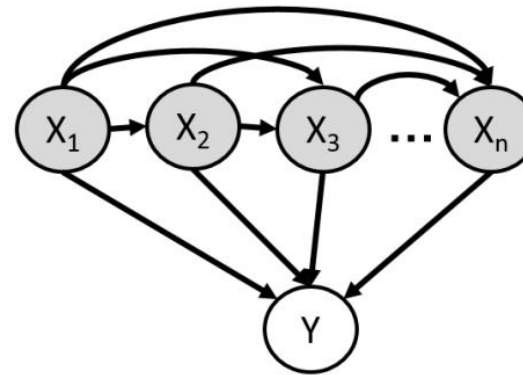
- ① Decision boundary $p(Y = 1 \mid \mathbf{x}; \boldsymbol{\alpha}) > 0.5$ is linear in $\mathbf{x}$
- ② Equal probability contours are straight lines
- ③ Probability rate of change has very specific form (third plot)

Discriminative models are powerful

Generative (Naïve Bayes)



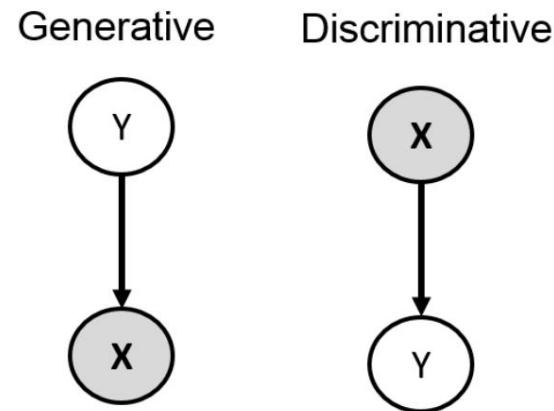
Discriminative (logistic regression)



- Logistic model does not assume $X_i \perp X_{-i} \mid Y$, unlike naive Bayes
- This can make a big difference in many applications
 - E.g., in spam classification, $X_1 = 1$ [“bank” in e-mail], $X_2 = 1$ [“account” in e-mail]
- Regardless of whether spam, these always appear together, i.e. $X_1 = X_2$
- Learning in naive Bayes results in $p(X_1 \mid Y) = p(X_2 \mid Y)$. Thus, naive Bayes double counts the evidence
- Learning with logistic regression sets $\alpha_1 = 0$ **or** $\alpha_2 = 0$, in effect ignoring it

Generative models are still very useful

- Using chain rule $p(Y, \mathbf{X}) = p(\mathbf{X} | Y)p(Y) = p(Y | \mathbf{X})p(\mathbf{X})$. Corresponding Bayesian networks:



- Using a discriminative model is only possible when $\mathbf{X}$ is always observed
- When some X_i variables are unobserved, the generative model allows us to compute $p(Y | \mathbf{X}_{\text{evidence}})$ by marginalizing over the unseen variables

Neural Models

- In discriminative models, we assume that

$$p(Y = 1 \mid \mathbf{x}; \alpha) = f(\mathbf{x}, \alpha)$$

- **Linear** dependence:

let $z(\alpha, \mathbf{x}) = \alpha_0 + \sum_{i=1}^n \alpha_i x_i$.

$p(Y = 1 \mid \mathbf{x}; \alpha) = \sigma(z(\alpha, \mathbf{x}))$, where $\sigma(z) = 1/(1 + e^{-z})$ is **logistic function**

Dependence might be too simple

- **Non-linear** dependence: let $\mathbf{h}(A, \mathbf{b}, \mathbf{x}) = f(A\mathbf{x} + \mathbf{b})$ be a non-linear transformation of the inputs (features).

$$p_{\text{Neural}}(Y = 1 \mid \mathbf{x}; \alpha, A, \mathbf{b}) = \sigma(\alpha_0 + \sum_{i=1}^h \alpha_i h_i)$$

More flexible

More parameters: $A, \mathbf{b}, \alpha$

Neural Models

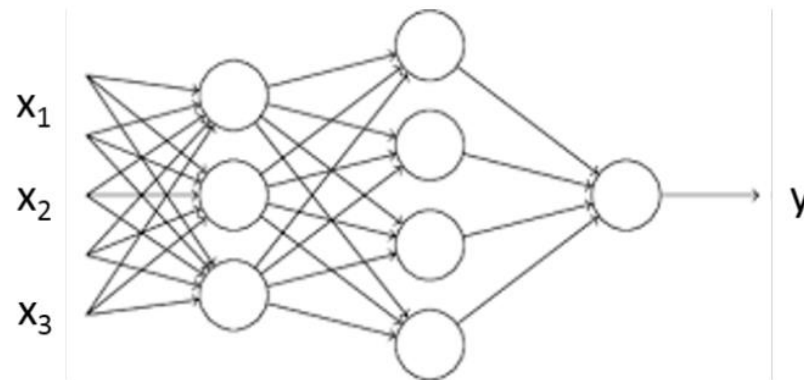
- In discriminative models, we assume that

$$p(Y = 1 \mid \mathbf{x}; \alpha) = f(\mathbf{x}, \alpha)$$

- Non-linear** dependence: let $\mathbf{h}(A, \mathbf{b}, \mathbf{x}) = f(A\mathbf{x} + \mathbf{b})$ be a non-linear transformation of the inputs (features).

$$p_{\text{Neural}}(Y = 1 \mid \mathbf{x}; \alpha, A, \mathbf{b}) = \sigma(\alpha_0 + \sum_{i=1}^h \alpha_i h_i)$$

Can repeat multiple times to get a neural network



Summary: Bayesian networks vs neural models

- Using Chain Rule (Fully General)

$$p(x_1, x_2, x_3, x_4) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2)p(x_4 \mid x_1, x_2, x_3)$$

- Bayes Net (Assumes conditional independencies)

$$p(x_1, x_2, x_3, x_4) \approx p(x_1)p(x_2 \mid x_1)p(x_3 \mid \cancel{x_1}, x_2)p(x_4 \mid \cancel{x_1}, \cancel{x_2}, x_3)$$

- Neural Models (Assume specific functional form for the conditionals. A sufficiently deep neural net can approximate any function.)

$$p(x_1, x_2, x_3, x_4) \approx p(x_1)p(x_2 \mid x_1)p_{\text{Neural}}(x_3 \mid x_1, x_2)p_{\text{Neural}}(x_4 \mid x_1, x_2, x_3)$$



Thank You!